

Universidade do Minho

Escola de Ciências

Licenciatura em Ciência de Dados

Daniel Arandas – A105990

Kássia Guimarães – A105755

Sara Marques – A85571

Vanessa Gomes – A105871

PROJETO EM CIÊNCIA DE DADOS

Agentic AI para agendamento hospitalar inteligente

Trabalho efetuado sob a orientação do

Docente Manuel Filipe Vieira Torres Santos e Tiago André Saraiva Guimarães, e da equipa: João António Araújo Lopes; Júlio Miguel Marques Duarte; Afonso Augusto Paula Lobo

Coordenador da UC: Luís Miguel Rocha Matos.

maio, 2026

Índice

Lista de Figuras.....	iii
Lista de Abreviaturas e Siglas.....	iv
RESUMO	v
ABSTRACT	vi
Introdução.....	1
Contextualização do problema	2
Motivação do projeto.....	2
Objetivos gerais e específicos	3
Enquadramento do Projeto	4
Agendamento hospitalar: desafios e importância	4
Utilizadores e serviços envolvidos	4
Critérios de sucesso do sistema	4
Limitações e riscos do contexto real	5
Metodologia	6
Abordagem do estudo.....	6
Recursos e equipa de trabalho.....	6
Tecnologias e ferramentas utilizadas	6
Arquitetura da solução	8
Implementação dos agentes	8
Integração com o motor de otimização	11
Componentes RAG.....	12
Implementação da camada conversacional	13
Regras de decisão e mecanismos de segurança.....	15
Geração e apresentação da proposta de solução.....	15
Avaliação da solução.....	19
Objetivos da avaliação	19
Cenários de teste	19
Métricas de avaliação	19
Resultados obtidos.....	20
Análise dos erros ou falhas	21
Trabalhos Futuros.....	22

Conclusão	23
Referências	24
Apêndice A – <i>Prompt</i> para avaliação das conversas	i

Lista de Figuras

Fig. 1. Exemplo de implementação do agente classificador e integração com a interface gráfica.	8
Fig. 2. Exemplo de implementação no n8n do agente de agendamentos, HTTP <i>request</i> do modelo de otimização e sistema RAG para atualização dos dados de agendamento.	9
Fig. 3. Exemplo de aplicação do agente de consulta à base de dados e sistema RAG para atualização dos dados de lista de espera.	9
Fig. 4. Exemplo da implementação do agente para alteração da base de dados e sistema RAG para alteração dos agendamentos.	10
Fig. 5. Exemplo de aplicação do agente de apoio a análises das bases de dados.	11
Fig. 6. Interface Web desenvolvida em forma de chat.	14
Fig. 7. Resultado da interface com a solicitação de novos agendamentos.	16
Fig. 8. Resultado da interface à consulta das últimas cirurgias agendadas.	17

Lista de Abreviaturas e Siglas

CSV – *Comma-Separated Values*

LLM – *Large Language Model*

IA – Inteligência Artificial

SNS – Serviço Nacional de Saúde

JSON – *JavaScript Object Notation*

HTTP – *Hypertext Transfer Protocol*

RESUMO

O agendamento de cirurgias continua a ser um desafio complexo, condicionado por regras clínicas, disponibilidade de recursos e prioridades institucionais. Os modelos de otimização existentes produzem agendas de alta qualidade, mas exigem entradas estruturadas e não oferecem uma interface em linguagem natural para os profissionais de saúde. Este projeto apresenta uma arquitetura multiagente suportada por *Large Language Models* (LLMs) que atua como intermediária entre os profissionais de saúde e os motores de otimização subjacentes. Desenvolvido com modelos alojados no Ollama, orquestrados através de *workflows* em n8n e suportados por um *backend* em Supabase para persistência estruturada de dados, o sistema proposto interpreta pedidos em linguagem natural, converte-os em parâmetros estruturados, invoca o modelo de otimização apropriado e comunica as propostas de agendamento resultantes ao utilizador em linguagem simples. A arquitetura proposta simplifica a interação com os sistemas de agendamento, reduzindo as barreiras de adoção e diminuindo a carga cognitiva associada a processos de tomada de decisão complexos. Ao permitir que clínicos e pessoal administrativo interajam com modelos de otimização através de uma conversa natural, o sistema democratiza o acesso a capacidades avançadas de agendamento sem necessidade de conhecimentos técnicos. No entanto, a implementação atual requer trabalho adicional, sobretudo nas áreas de validação de resultados e afinação dos modelos, para mitigar alucinações e outras limitações inerentes a sistemas baseados em LLM em ambientes de saúde críticos para a segurança.

Palavras-chave: *Large Language Models* – Agendamento Hospitalar Inteligente – Agentic AI.

ABSTRACT

Hospital surgery scheduling remains a complex challenge, constrained by clinical rules, resource availability, and institutional priorities. Existing optimization models produce high-quality schedules but require structured inputs and offer no natural-language interface for medical professionals. This project presents a multi-agent architecture powered by Large Language Models (LLMs) that acts between healthcare professionals and underlying optimization engines. Built using Ollama-hosted models, orchestrated through n8n workflows, and supported by a Supabase backend for structured data persistence, the proposed system interprets natural-language requests, converts them into structured parameters, invokes the appropriate optimization model, and communicates the resulting scheduling proposals back to the user in plain language. The proposed architecture simplifies the interaction with scheduling systems, lowering adoption barriers and reducing the cognitive load associated with complex decision-making processes. By enabling clinicians and administrative staff to engage with optimization models through natural conversation, the system democratizes access to advanced scheduling capabilities without requiring technical expertise. Nevertheless, the current implementation requires further work, particularly in the areas of output validation and model fine-tuning, to address hallucinations and other inherent limitations of LLM-based systems in safety-critical healthcare environments.

Keywords: Large Language Models – Smart Hospital Scheduling – Agentic AI.

Introdução

Este projeto propõe o desenvolvimento de uma solução de Agentic AI ¹ especificamente desenhada para apoiar a equipa médica na gestão do agendamento hospitalar. Partindo dos desafios atuais de planeamento cirúrgico, como a existência de listas de espera extensas, a necessidade de cumprir restrições operacionais e a elevada carga administrativa associada à marcação e organização de cirurgias, a solução recorre a uma arquitetura multiagente para automatizar parte deste processo, mantendo a segurança e o controlo por parte dos profissionais de saúde.

A arquitetura desenvolvida utiliza agentes especializados para converter pedidos complexos, formulados em linguagem natural por profissionais de saúde, em planos de agendamento otimizados com base em modelos de otimização existentes. Estes agentes são responsáveis por interpretar regras institucionais, consultar e atualizar informação sobre listas de espera e agendamentos, e articular-se com componentes de recuperação de informação (RAG) para garantir que as decisões são tomadas com base em dados atualizados e clinicamente relevantes.

O foco central reside na criação de um assistente inteligente capaz de atuar como interface conversacional entre os utilizadores e o sistema de agendamento, facilitando a tomada de decisão clínica e de gestão. Ao reduzir a necessidade de interações manuais com ferramentas técnicas e ao automatizar tarefas de consulta, marcação e alteração de cirurgias, o trabalho pretende contribuir para aumentar a eficiência e diminuir a carga administrativa no contexto do Serviço Nacional de Saúde, potenciando uma gestão mais transparente e informada.

¹ Sistema de IA autónomo que pode agir de forma independente para atingir objetivos predeterminados, proativo e pode executar tarefas complexas sem supervisão humana constante. “Agentic” indica agência – a capacidade desses sistemas de agir independentemente, mas de maneira orientada a objetivos [1].

Contextualização do problema

Motivação do projeto

Dados divulgados pela Health News [2] indicam a instabilidade da saúde pública em Portugal ao revelar o agravamento das listas de espera no Serviço Nacional de Saúde (SNS). Em 2025, a Lista de Espera para Consulta aumentou 13,8% e a Lista de Inscritos para Cirurgia cresceu 3,4%, totalizando, no final do ano, mais de 1,08 milhão de utentes à espera de consulta e 264 mil à espera de cirurgia. Esses números mostram que, embora o SNS tenha mantido atividade elevada, a sua capacidade de resposta permanece insuficiente para atender à procura crescente, comprometendo a eficiência, a rapidez no atendimento e o acesso oportuno da população aos cuidados de saúde.

Perante esse cenário, a Inteligência Artificial (IA) tem vindo a transformar rapidamente o setor da saúde, possibilitando avanços no diagnóstico, na gestão de pacientes e na eficiência operacional. Ao automatizar tarefas administrativas, otimizar o agendamento de procedimentos hospitalares e apoiar a previsão de necessidades hospitalares, a IA pode contribuir para reduzir tempos de espera e melhorar a gestão dos recursos, apresentando-se como uma alternativa relevante para enfrentar parte das fragilidades observadas no sistema de saúde [3].

Neste contexto, a utilização de sistemas de inteligência artificial com capacidade conversacional surge como uma oportunidade para melhorar a interação entre utilizadores e sistemas de marcação, permitindo transformar pedidos de linguagem natural em instruções estruturadas e acionáveis. No entanto, em ambiente hospitalar, essa intermediação não pode limitar-se à geração de respostas automáticas, sendo necessário garantir correção clínica, segurança na comunicação e respeito pelas regras de negócio da instituição.

O presente projeto insere-se precisamente neste desafio, propondo o desenvolvimento de uma arquitetura multiagente baseada em *Large Language Models*² (LLMs), capaz de funcionar como intermediária entre o utilizador e os modelos de otimização de agendamento já existentes.

² Categorias de modelos de aprendizagem profunda treinados com imensas quantidades de dados, que os torna capazes de compreender e gerar linguagem natural e outros tipos de conteúdo para executar uma ampla quantidade de tarefas [4].

Objetivos gerais e específicos

Com este projeto, pretende-se que o sistema seja capaz de interpretar pedidos, aplicar restrições e prioridades, invocar corretamente os modelos de otimização e comunicar ao utilizador propostas de agendamento claras, contextualizadas e justificadas.

Desta forma, o problema central do projeto não reside apenas na automatização do agendamento hospitalar, mas na criação de uma solução inteligente e fiável que combine capacidade conversacional, suporte à decisão e integração com mecanismos formais de otimização. A relevância deste problema justifica-se pelo potencial de melhoria na eficiência operacional, na consistência das decisões e na qualidade da comunicação com o utilizador final.

Enquadramento do Projeto

Agendamento hospitalar: desafios e importância

O agendamento hospitalar constitui um processo crítico na gestão de serviços de saúde, uma vez que envolve a articulação entre disponibilidade de recursos, prioridades clínicas e necessidades dos utentes. Em contextos reais, este processo tende a ser complexo, dinâmico e sujeito a múltiplas restrições, o que dificulta a tomada de decisão rápida e consistente.

Com a chegada da inteligência artificial, a gestão hospitalar está a otimizar as operações através da automação de tarefas administrativas, o agendamento de procedimentos cirúrgicos e da gestão de recursos hospitalares, conseguindo-se, assim, não só reduzir os custos operacionais, como também melhorar a experiência do paciente, ao reduzir os tempos de espera e aumentar a eficiência dos processos hospitalares [3].

Utilizadores e serviços envolvidos

Os principais utilizadores do sistema proposto são os médicos cirurgiões e os chefes de departamentos hospitalares, responsáveis pela proposta, validação e gestão das cirurgias eletivas.

Para além destes utilizadores diretos, o sistema impacta serviços de bloco operatório e equipas de apoio, em que a organização do trabalho depende das agendas cirúrgicas. A qualidade do agendamento reflete-se ainda na experiência dos doentes, nomeadamente nos tempos de espera, no número de remarcações e na previsibilidade do percurso cirúrgico.

Critérios de sucesso do sistema

A inteligência artificial pode apoiar o agendamento inteligente no setor da saúde ao prever procura, faltas, atrasos, tempos de atendimento e tempos de espera, permitindo organizar melhor as agendas e utilizar os recursos hospitalares de forma mais eficiente. Em vez de reagir aos problemas depois de estes ocorrerem, os modelos de IA permitem uma gestão mais proativa das consultas, exames, cirurgias e fluxos de pacientes [5].

No contexto deste projeto, o sucesso do sistema será avaliado pela sua capacidade de assegurar uma interação conversacional fiável e segura, interpretando pedidos em linguagem natural e convertendo-os em parâmetros estruturados de forma clinicamente correta e em conformidade com as regras

institucionais. Além disso, considera-se essencial que o agente tome decisões de agendamento corretas e contextualizadas, aplicando restrições, prioridades e regras de negócio de forma consistente.

Outro critério de sucesso relevante é a capacidade do sistema para invocar e executar corretamente os modelos de otimização já existentes, garantindo a passagem adequada de *inputs* e a interpretação rigorosa dos *outputs*. Do mesmo modo, o sistema deverá comunicar de forma clara a proposta de agendamento ao utilizador, apresentando as marcações sugeridas, alternativas possíveis e respetivas justificações de forma compreensível.

Por fim, o projeto será igualmente bem-sucedido se resultar numa arquitetura multiagente modular e escalável, que facilite a separação de responsabilidades, a manutenção e a evolução futura da solução.

Limitações e riscos do contexto real

Num contexto real hospitalar, uma das principais limitações está na necessidade de garantir que a integração conversacional seja fiável, segura e clinicamente correta, porque qualquer erro de interpretação de pedidos em linguagem natural pode comprometer a qualidade dos agendamentos. Isso significa que o sistema pode enfrentar problemas com pedidos incompletos ou mal formulados.

Outro risco importante está relacionado com a tomada de decisão de agendamento, já que o agente precisa aplicar restrições, prioridades e regras de forma consistente num ambiente real, onde os cenários tendem a ser mais complexos e variáveis. Em contexto hospitalar, pequenas falhas nessa decisão podem originar marcações inadequadas, conflitos de recursos ou propostas pouco adequadas à situação clínica.

Metodologia

Abordagem do estudo

O projeto centra-se no desenvolvimento de um agente de inteligência artificial conversacional para apoio ao agendamento hospitalar.

Para tal, recorreu-se a um ambiente de desenvolvimento numa máquina virtual ³, onde foram integradas as principais componentes tecnológicas: uma base de dados PostgreSQL [6], a plataforma n8n [7] para a criação de fluxos e coordenação de chamadas entre serviços, modelos de linguagem em grande dimensão (LLMs) executados através do Ollama [8] para a criação do agente conversacional e uma interface ⁴ desenvolvida em Streamlit [9] para interação com os utilizadores.

Recursos e equipa de trabalho

Ao nível dos recursos humanos, o projeto conta com cinco docentes orientadores e quatro estudantes da Licenciatura em Ciências de Dados. No que respeita aos recursos tecnológicos, o trabalho recorre a computadores pessoais dos elementos da equipa e a uma máquina virtual disponibilizada pelos orientadores, na qual são instaladas e executadas as principais componentes da solução.

Esta infraestrutura permite concentrar o ambiente de desenvolvimento e facilitar a gestão dos serviços para o funcionamento do agente de IA e dos sistemas de apoio ao agendamento hospitalar.

Tecnologias e ferramentas utilizadas

A principal ferramenta utilizada para a criação da arquitetura de Agentic AI foi o n8n [7], uma plataforma de automação que conecta aplicações, bases de dados, entre outros serviços através de fluxos visuais. Trata-se de uma plataforma de código aberto que facilita a integração entre serviços e aplicações, permitindo a criação de fluxos de trabalho automatizados e personalizados [12]. No contexto do projeto,

³ Ambiente virtual que funciona como um computador físico, permitindo executar um sistema operativo e aplicações de forma independente. A máquina virtual utiliza parte dos recursos do computador ou servidor físico, como processador, memória e armazenamento, que são geridos por *software* de virtualização. Esta tecnologia permite executar vários ambientes virtuais, com diferentes sistemas operativos, num mesmo dispositivo físico [10].

⁴ Camada visual e interativa de um site, aplicação ou *software*, engloba todos os elementos visuais e funcionais com os quais os utilizadores interagem. O objetivo principal é proporcionar uma experiência de utilizador intuitiva e agradável, tornando a navegação e a interação com o produto o mais simples e eficaz possível [11].

o n8n é utilizado para estabelecer uma ligação entre a interface e a base de dados, assim como para administrar as interações com o modelo conversacional (*chat model*).

A interface de utilizador foi desenvolvida em Streamlit [9], uma biblioteca de código aberto em *Python* que permite a criação de aplicações *Web* interativas, onde foi implementado um *chat* para apoio na gestão de agendamentos hospitalares. A aplicação foi criada numa nuvem do Streamlit, sendo o código-fonte disponibilizado num repositório GitHub [13], [24].

A base de dados utilizada no projeto foi disponibilizada pelos orientadores no formato de *Comma-Separated Values*⁵ (CSV), contendo informações para o funcionamento do agente, tanto ao nível de treino e configuração como das operações de consulta da lista de espera de cirurgias hospitalares.

O *chat model* foi desenvolvido com modelos disponibilizados pelo Ollama [8], uma ferramenta de código aberto que executa LLMs localmente. Esta abordagem contribui para maior controlo sobre os dados transmitidos e reduz potenciais riscos de segurança associados ao armazenamento em nuvem. Além disso, essa ferramenta de IA *offline* também ajuda a reduzir a latência e a dependência de servidores externos, tornando-as mais rápidas e fiáveis [14].

⁵ Ficheiro de texto que armazena dados tabulares, como números e texto, em colunas separadas por vírgulas, permitindo a sua fácil importação para diferentes sistemas de gestão de dados.

Arquitetura da solução

Implementação dos agentes

Foi implementada uma arquitetura multiagente, em que cada agente desempenha uma função específica no processo de apoio ao agendamento hospitalar, permitindo segmentar tarefas e aumentar a eficiência face ao problema em estudo.

O Agente Classificador é responsável por interpretar o pedido inicial do utilizador e encaminhá-lo para o fluxo adequado, conforme representado na Fig. 1. Para tal, foi definido um *prompt*⁶ especializado que instrui o modelo a analisar a mensagem do utilizador e a responder num formato *JavaScript Object Notation*⁷ (JSON). Essa resposta contém a mensagem de entrada do utilizador, permitindo ao mecanismo *Switch* identificar a intenção associada ao pedido. Quando a intenção é “agendar”, o pedido é encaminhado para o agente de agendamentos; quando se trata de uma “consulta de informação”, é encaminhado para o agente de consultas; se o objetivo for “alterar o agendamento” a mensagem é encaminhada para o agente de alterações; se a mensagem for vaga ou incompleta, o agente pode gerar perguntas adicionais ao utilizador, de forma a contextualizar o pedido antes de encaminhar.

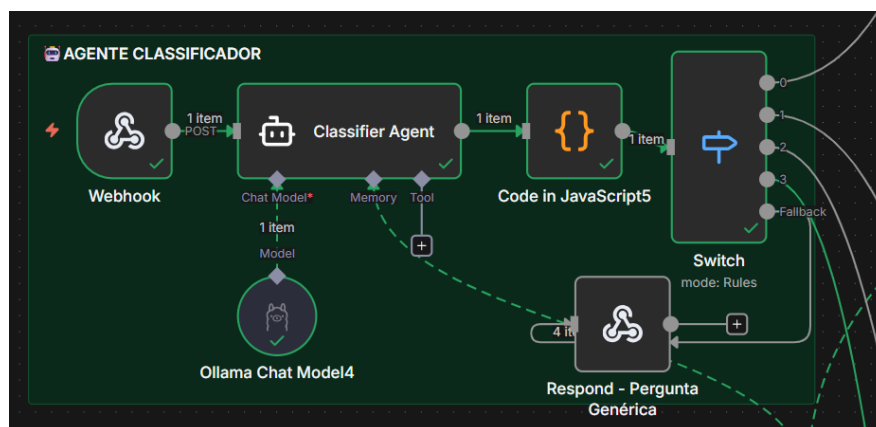


Fig. 1. Exemplo de implementação do agente classificador e integração com a interface gráfica.

O Agente de Agendamentos atua como um assistente dedicado à marcação das cirurgias. Sempre que o pedido corresponder a um agendamento, este agente recebe a mensagem e conduz o utilizador no processo de marcação, procurando extrair informações sobre as datas pretendidas. Caso a data não seja fornecida ou válida para a estrutura de saída, o agente solicita essa informação ao utilizador, garantindo

⁶ Texto de linguagem natural que solicita que a Inteligência Artificial execute uma tarefa específica. O processo de engenharia de *prompt* orienta as soluções da inteligência artificial para gerar os resultados desejados, onde requer instruções detalhadas para criar resultados relevantes e de alta qualidade [15].

⁷ Formato de transação de dados leve, uma linguagem em que os computadores e as aplicações usam para comunicar e partilhar informação [16].

que os parâmetros mínimos para entrada no motor de otimização sejam válidos para prosseguir, conforme representado na Fig. 2. Após a validação e processamento do pedido, o agente procede ainda à atualização da base de dados de agendamentos.

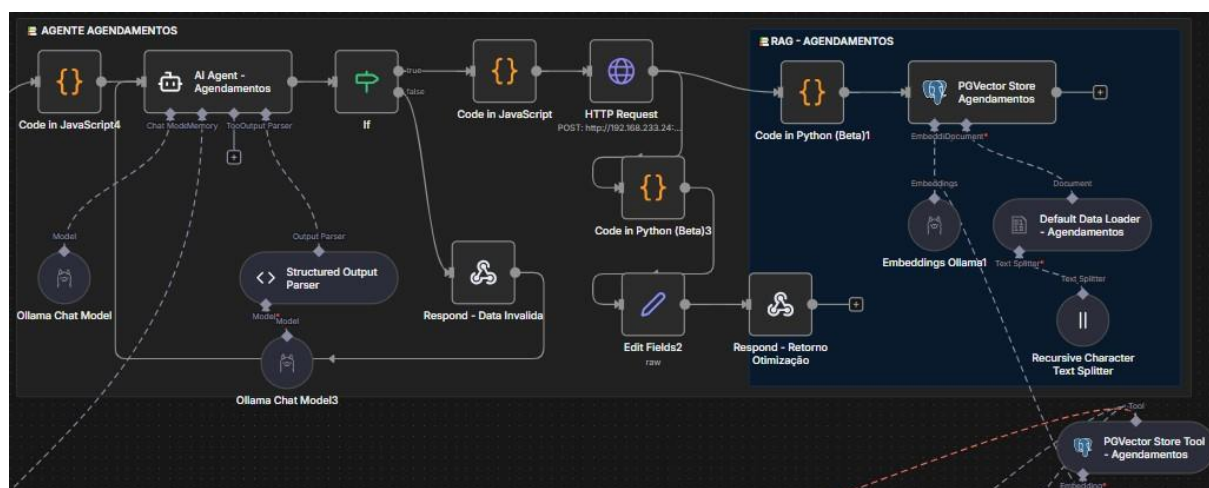
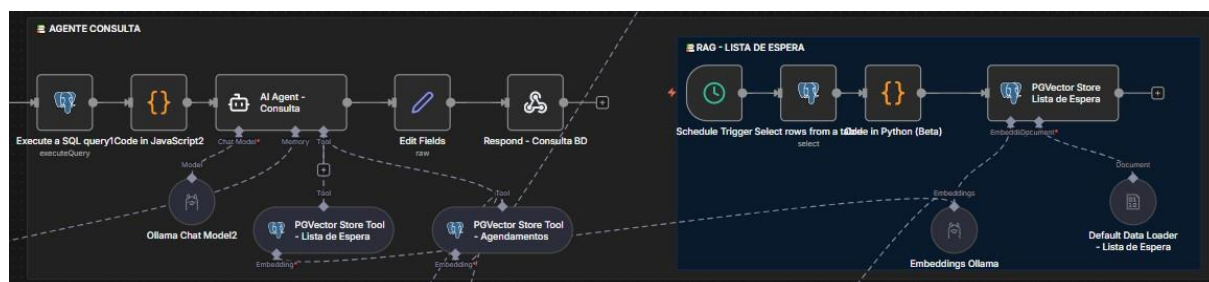


Fig. 2. Exemplo de implementação no n8n do agente de agendamentos, HTTP *request* do modelo de otimização e sistema RAG para atualização dos dados de agendamento.

Por sua vez, o Agente de Consultas é responsável por responder a pedidos de consulta de informação, referente à lista de espera de cirurgias e a agendamentos já realizados. Quando o utilizador pretende, por exemplo, saber o estado de uma lista de espera, o pedido é encaminhado para este agente, que recorre a um sistema *Retrieval Augmented Generation* (RAG), uma técnica que aprimora LLMs integrando fontes de dados externas [17]. Tecnicamente, o sistema baseia-se em bases de dados vetoriais, implementadas através de `PGVector` em PostgreSQL, que permite realizar pesquisas sobre os dados de listas de espera e agendamentos, conforme representado na Fig. 3. O agente consulta esta base de dados vetorial para obter o contexto mais relevante e, em seguida, gera uma resposta em que apresenta a informação solicitada de forma compreensível ao utilizador.



O Agente de Alterações foi concebido para, sempre que solicitado, realizar modificações nos agendamentos já armazenados na base de dados vetorial do sistema RAG. Este agente permite ao utilizador remover turnos, alterar ou cancelar uma cirurgia, bem como modificar a duração de um turno previamente definido.

O principal papel deste agente consiste em interagir com o utilizador de forma dinâmica, conduzindo um diálogo até compreender com precisão a intenção da alteração pretendida. Após identificar todas as modificações necessárias, o agente apresenta ao utilizador um resumo completo das alterações propostas e solicita a respetiva confirmação. Depois da confirmação por parte do utilizador, o sistema envia os dados já estruturados e formatados para o `PGVector Store` em PostgreSQL, inserindo uma nova versão atualizada do agendamento na base de dados vetorial do sistema RAG, como podemos visualizar na Fig. 4. Desta forma, sempre que o Agente de Consultas realizar uma pesquisa relacionada com o agendamento, a informação devolvida corresponderá à versão mais recente e atualizada dos dados.

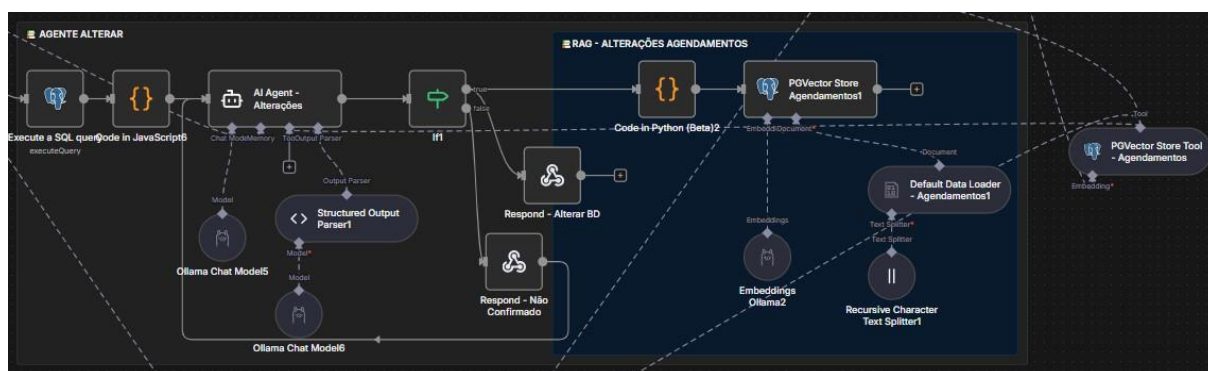


Fig. 4. Exemplo da implementação do agente para alteração da base de dados e sistema RAG para alteração dos agendamentos.

Por fim, foi desenvolvido um Agente de Análises – concebido como uma especialização do agente de consulta – com o objetivo de apoiar a extração de métricas a partir das bases de dados de listas de espera e de agendamentos. Ao contrário do Agente de Consultas, que se foca na resposta a pedidos diretos de informação (como o estado da lista de espera para determinada especialidade ou os detalhes de uma cirurgia específica), o Agente de Análises está orientado para pedidos quantitativos e agregados, como contagens por turno, por data, número de doentes por médico ou distribuição de cirurgias por especialidade. Esta separação de responsabilidades foi introduzida para reduzir o risco de alucinações por parte do agente de Consultas em questões mais complexas e para manter cada agente focado num

conjunto bem definido de tarefas. No *workflow* implementado em n8n, este agente é acionado quando o pedido do utilizador envolve expressões como “quantos” ou “quantidade de”, permitindo encaminhar este tipo de pergunta para um fluxo especializado de análise. O fluxo de trabalho correspondente a este agente, implementado no n8n, pode ser observado na Fig. 5.

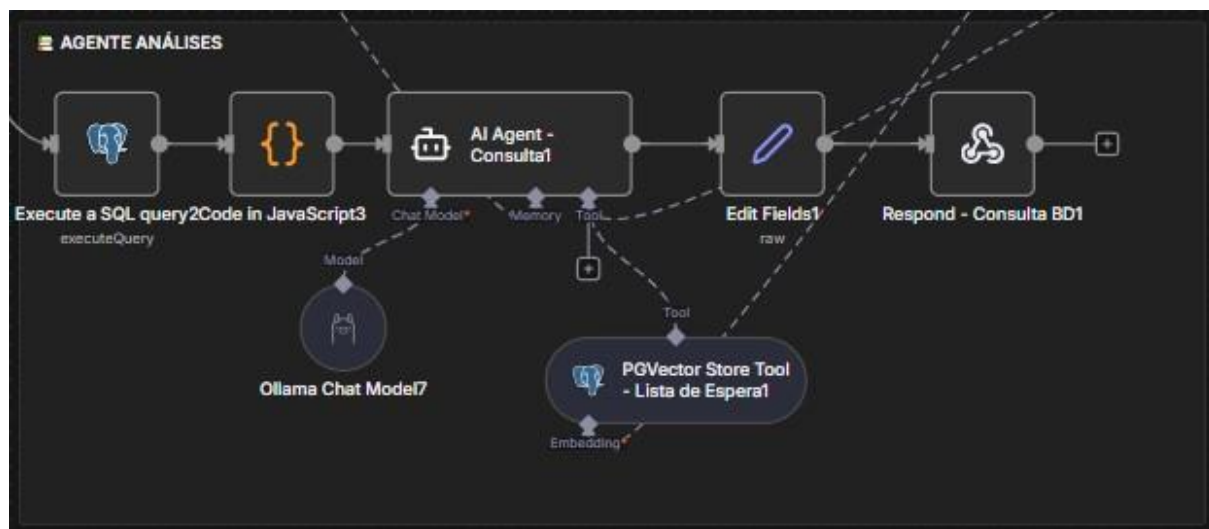


Fig. 5. Exemplo de aplicação do agente de apoio a análises das bases de dados.

Integração com o motor de otimização

O modelo de otimização foi disponibilizado pelos professores orientadores e é invocado através de um pedido *Hypertext Transfer Protocol* (HTTP), onde é enviado um JSON com os parâmetros de datas, obtidos pelo Agente de Agendamentos, conforme representado na Fig. 2. Com base na entrada, o motor realiza a otimização considerando várias fontes de dados, como os turnos dos profissionais, as listas de espera e as cirurgias já agendadas.

A partir dos dados da lista de espera e da informação sobre a disponibilidade de turnos, o modelo realiza a alocação otimizada das cirurgias aos turnos existentes, distribuindo-as por dia e turno em função das restrições e critérios definidos. Como resultado, o motor de otimização devolve um JSON, com diversas informações, como as cirurgias alocadas e respetivos turnos, informações sobre o número de cirurgias agendadas, o número de cirurgias ainda em espera após as alocações realizadas, a percentagem de utilização dos serviços que foi aplicada, penalidade total, uma variável sobre qualidade da solução dada pelo modelo de otimização, e quantidade de turnos desrespeitados pela alocação das cirurgias.

Componentes RAG

O RAG é uma técnica que aprimora os LLMs, integrando-os com fontes de dados externas, resolvendo algumas limitações de grandes modelos, como conhecimento desatualizado, alucinações e respostas genéricas. Ao combinar os recursos de geração de modelos com mecanismos precisos de recuperação de informação, o RAG permite que sistemas de IA produzam respostas mais precisas e contextualmente relevantes.

No processo, os dados necessários são reunidos, divididos em blocos de texto menores, transformados e convertidos em uma representação vetorial através de um modelo de *embeddings*. Estes *embeddings* correspondem a representações numéricas que capturam o significado semântico por trás do texto. Os blocos de texto recuperados da pesquisa semântica, juntamente com a consulta inicial do utilizador, são enviados ao LLM. Em seguida, o algoritmo utiliza essas informações para gerar uma resposta coerente às perguntas do utilizador [17].

No contexto deste projeto, foram implementados três componentes RAG:

1. **RAG para Agendamentos:** com os dados que saem do motor de otimização, correspondentes aos agendamentos efetuados. Numa fase inicial, foi implementado um código em linguagem Python responsável por transformar os dados recebidos num formato mais apropriado para a criação de uma base de dados vetorial com o Supabase [18]. Após essa etapa de pré-processamento, os dados são convertidos em *embedding*, com recurso ao modelo `nomic-embed-text:latest` e inseridos na tabela vetorial agendamentos, através do nodo `PGVector Store`. Posteriormente, essa mesma tabela é consultada pelos agentes através do nodo `PGVector Store Tool - Agendamentos`, configurado como ferramenta de recuperação de informação. Este fluxo encontra-se exemplificado na Fig. 2.
2. **RAG para Listas de Espera:** foi concebido para atualizar periodicamente a informação relativa às listas de espera. Para isto, foi criado um procedimento automático, acionado uma vez por dia através de um `Schedule Trigger`, que realiza uma consulta à base de dados PostgreSQL. A cada execução do *workflow*, os dados são recolhidos da tabela operacional, processados em Python, convertidos para um formato compatível com a base vetorial no Supabase e transformados em *embeddings* com recurso ao Ollama. Em seguida, os registos vetorizados são

inseridos na tabela vetorial `documentos`, ficando posteriormente disponíveis para consulta através do nodo `PGVector Store Tool - Lista de Espera`, utilizada pelo agente. Este processo permite que as respostas geradas reflitam a informação mais recente disponível na base de dados. É possível ver um exemplo deste fluxo na Fig. 3.

3. **RAG para Alterações:** desenvolvido para suportar alterações em agendamentos já existentes no sistema. Neste caso, o Agente de Alterações consulta inicialmente a informação armazenada na base vetorial do sistema RAG e apresenta ao utilizador os dados do agendamento a modificar. Após a indicação da alteração pretendida, o agente gera uma pré-visualização da nova configuração e solicita confirmação antes de qualquer registo ser incorporado na base vetorial. Quando existe a confirmação por parte do utilizador, os dados atualizados são processados através de código em Python, sendo formatados para integração na tabela vetorial de agendamentos. Em seguida, o processo segue uma lógica semelhante aos componentes anteriores: a nova versão textual do agendamento é convertida em *embeddings* com recurso ao Ollama e inserida no `PGVector Store`. Esta operação permite atualizar funcionalmente o repositório de conhecimento do sistema RAG, uma vez que a base passa a incorporar a versão mais recente do agendamento. Assim, futuras consultas podem recuperar a informação atualizada a partir da base vetorial consultada pelos agentes.

Implementação da camada conversacional

A camada conversacional do sistema foi implementada através de uma interface Web desenvolvida em Streamlit, permitindo ao utilizador interagir com o agente em forma de *chat*, conforme representado na Fig. 6. Nesta interface, o utilizador pode escrever perguntas livremente ou selecionar sugestões de questões frequentes, como consultar listas de espera, verificar cirurgias recentes ou solicitar novos agendamentos.



Fig. 6. Interface Web desenvolvida em forma de chat.

A aplicação apresenta o histórico da conversa de forma contínua, distinguindo visualmente as mensagens do utilizador das respostas do sistema. Sempre que a resposta envolve resultados estruturados, como listas de cirurgias ou indicadores de desempenho, esses elementos são apresentados sob a forma de tabelas e pequenos painéis de métricas, facilitando a interpretação da informação pelo utilizador.

Para estabelecer a ligação entre a interface e a arquitetura multiagente, foi utilizada uma integração baseada em *Webhook*, isto é, um mecanismo de comunicação orientado a eventos que envia dados automaticamente entre aplicações via HTTP. Sempre que o utilizador envia uma mensagem no *chat*, o Streamlit envia o conteúdo dessa mensagem para o *Webhook* configurado, que por sua vez encaminha o pedido para o fluxo implementado no n8n, onde está a arquitetura multiagente, conforme apresentado na Fig. 1. Após o processamento pelos agentes, a resposta gerada é devolvida e apresentada na interface conversacional, completando o ciclo de interação.

Regras de decisão e mecanismos de segurança

Regras de decisão e mecanismos de segurança são fundamentais para garantir que o sistema de agendamento atue de forma correta e alinhada com as restrições do contexto hospitalar. Para tal, foram desenvolvidos controlos específicos ao nível da classificação dos pedidos, da validação dos parâmetros e tratamento das respostas, reduzindo o impacto dos erros de entrada, falhas de informação e interpretações indevidas por parte dos agentes.

As entradas do sistema passam, numa primeira fase, por uma camada de decisão responsável por encaminhar corretamente os pedidos para o fluxo adequado, nomeadamente: agendamento, consulta, alteração ou outras operações. Inicialmente, a mensagem enviada pelo utilizador é processada pelo Agente Classificador (*Classifier Agent*), que interpreta o pedido e devolve uma estrutura em JSON com a mensagem de entrada e a resposta associada. Em seguida, o nodo `Switch` analisa o conteúdo textual da mensagem, recorrendo a padrões definidos para identificar expressões associadas às principais intenções do sistema (por exemplo, “agendar”, “alterar” ou “consultar”). Com base nessa verificação, o pedido é encaminhado para o agente especializado correspondente. Esta combinação entre interpretação linguística e regras explícitas de encaminhamento permite tornar o fluxo mais controlado e reduzir o risco de tratamento incorreto das mensagens.

Outro mecanismo de segurança, ao nível do Agente de Agendamentos, é o impedimento do processo de agendamento sem a indicação de um intervalo de datas válido. Sempre que essa informação não é fornecida ou não respeita o formato esperado, o agente solicita novamente ao utilizador que indique as datas pretendidas para agendamento, garantindo que a estrutura de saída em JSON cumpra os requisitos mínimos para ser enviada ao motor de otimização. Isto permite que o agente funcione de modo defensivo para possíveis falhas ou faltas de informação.

Os dados retornados pelo motor de otimização são produzidos num formato JSON, contendo muitos campos técnicos. Para apresentação ao utilizador, foi desenvolvido um código em Python que processa esse JSON e ajusta a informação a um formato legível, selecionando e reorganizando os elementos relevantes para a resposta final apresentada na interface conversacional.

Geração e apresentação da proposta de solução

A geração da proposta de solução assume diferentes formas consoante o tipo de pedido do utilizador, abrangendo desde a criação de novos agendamentos até à consulta e análise de informação existente.

No caso dos pedidos de agendamento, o processo é suportado pelo modelo de otimização, que recebe os parâmetros preparados pelo Agente de Agendamentos e calcula uma solução de alocação de cirurgias. O resultado é devolvido num objeto JSON com informação detalhada sobre cirurgias alocadas, turnos correspondentes, cirurgias que permanecem em espera e percentagem de utilização de recursos.

Esta resposta técnica é depois transformada numa proposta compreensível para o utilizador, conforme apresentado na Fig. 7. Os dados são selecionados e reorganizados, destacando elementos como a distribuição das cirurgias, o impacto na lista de espera e métricas globais do plano de agendamento. A interface recorre a painéis de métricas, nos quais, por exemplo, valores de utilização de recursos superiores a 90% são destacados a vermelho, chamando a atenção para situações de elevada carga operacional.

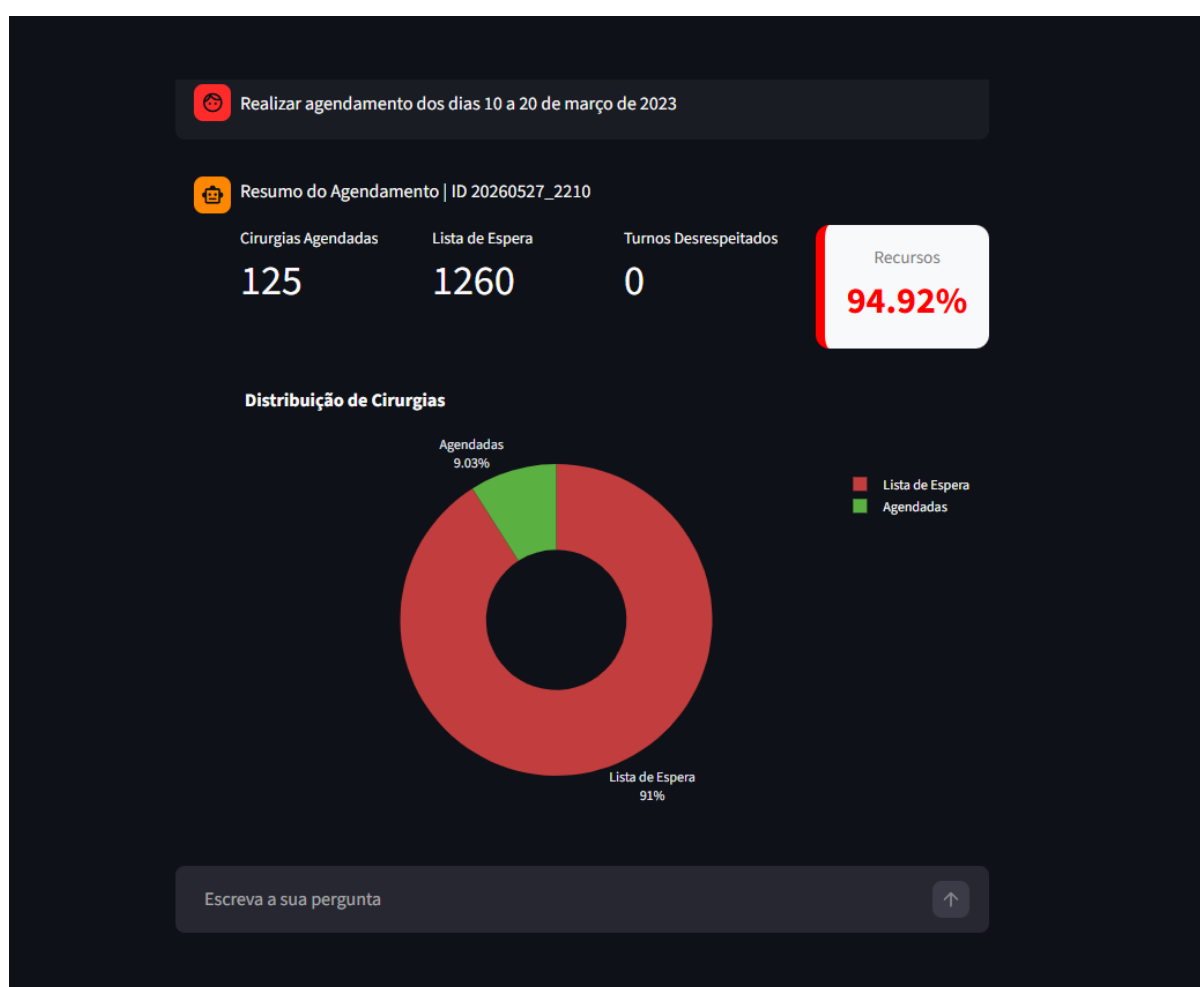


Fig. 7. Resultado da interface com a solicitação de novos agendamentos.

Para pedidos de consulta, alteração e análise de informação, a proposta de solução é gerada a partir dos agentes específicos, em articulação com os componentes RAG. No caso das consultas, o sistema

recupera o estado atual das listas de espera e dos agendamentos, devolvendo ao utilizador respostas que esclarecem, por exemplo, a situação de uma determinada especialidade ou de um conjunto de cirurgias. No caso das alterações, o agente apresenta uma síntese das modificações propostas e solicita confirmação antes de atualizar os dados. Após a confirmação, a resposta final inclui a nova configuração do agendamento. Já o Agente de Análises responde a pedidos quantitativos simples, como número de doentes por médico ou distribuição de cirurgias por turno, apresentando essas métricas em texto e em elementos visuais que apoiam a interpretação dos resultados. Todos esses resultados são apresentados na interface em formato textual e, quando aplicável, sob a forma de tabelas, facilitando a interpretação das informações pelo utilizador – como exemplificado na Fig. 8.

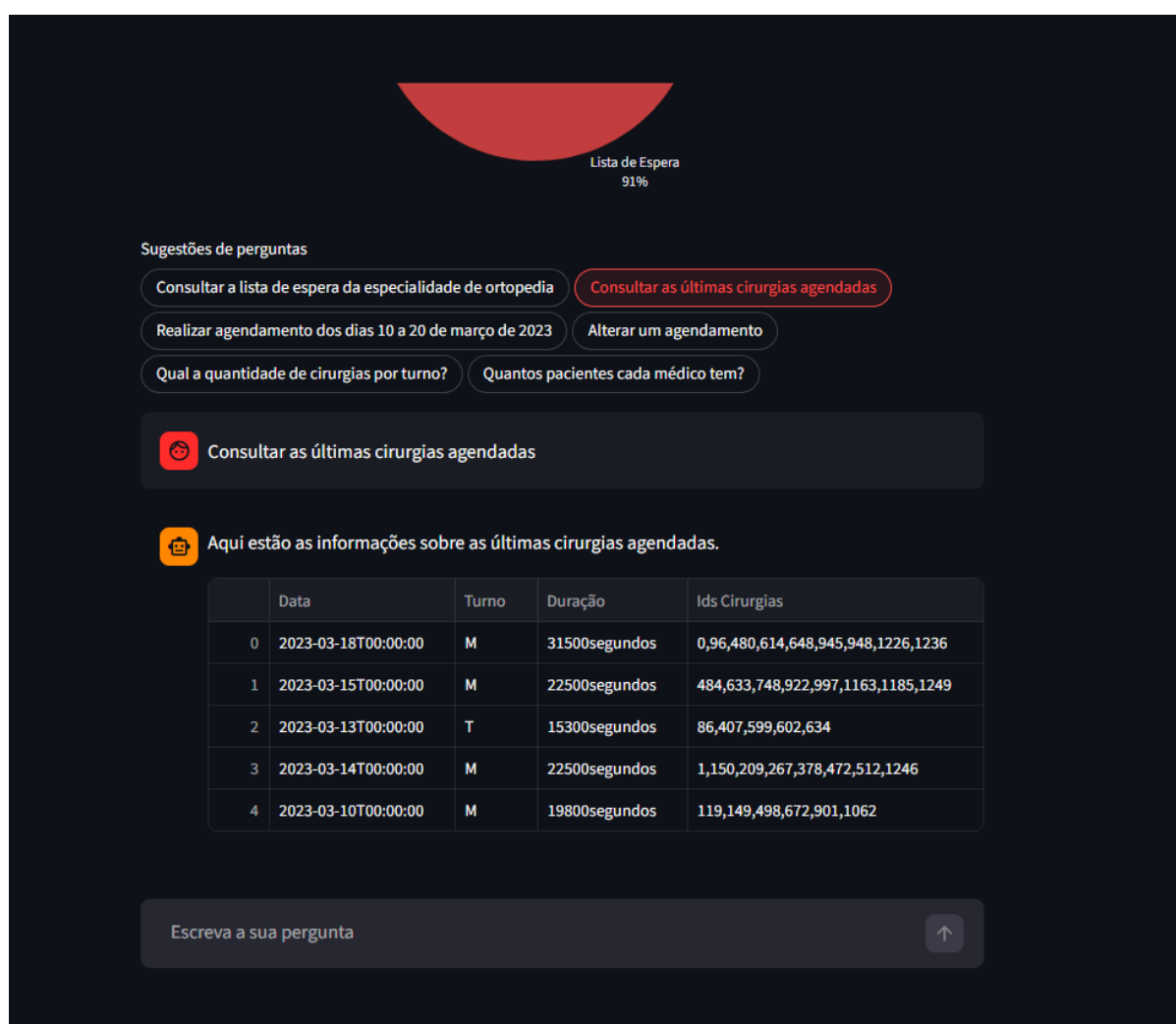


Fig. 8. Resultado da interface à consulta das últimas cirurgias agendadas.

Em todos os casos, a proposta de solução é apresentada na camada conversacional, através da interface em Streamlit, com as mensagens no chat complementadas, sempre que adequado, por tabelas e painéis.

Esta apresentação integrada permite ao utilizador compreender não só a resposta imediata ao seu pedido, mas também o contexto e o impacto da solução gerada pelo sistema.

Avaliação da solução

Objetivos da avaliação

A avaliação da solução teve como principal objetivo analisar a capacidade do sistema multi-agente em interpretar corretamente pedidos em linguagem natural, bem como o seu desempenho na geração de respostas úteis para o utilizador. Em particular, pretendeu-se medir a qualidade das interações com os diferentes agentes (Agendamentos, Consultas, Alterações e Análises), avaliando a exatidão das respostas face aos dados disponíveis e o cumprimento das regras definidas para o contexto hospitalar. Adicionalmente, foi considerado o tempo de resposta como indicador da viabilidade prática da solução, dado o impacto que a latência pode ter na adoção do sistema em cenários reais de utilização.

Cenários de teste

Para a avaliação do sistema foram definidos cenários de teste baseados em dados reais de listas de espera e agendamentos de cirurgias, utilizados tanto pelos modelos de otimização como pelos componentes RAG. Estes cenários procuram refletir situações próximas de um ambiente hospitalar, incluindo listas de espera extensas, múltiplas especialidades, diferentes combinações de turnos e recursos disponíveis.

As interações testadas abrangeram pedidos de novos agendamentos, consultas ao estado das listas de espera e dos agendamentos existentes, solicitações de alteração de cirurgias e pedidos de análise sobre a base de dados. Desta forma, foi possível observar o comportamento do sistema em tarefas centrais do processo de gestão de cirurgias.

Métricas de avaliação

Trabalhos recentes demonstram que modelos de linguagem de grande dimensão podem ser utilizados como mecanismos de avaliação automática de sistemas generativos ("*LLM as a judge*"), apresentando elevada correlação com avaliadores humanos [19].

Desta forma, para avaliar a qualidade das interações com os diferentes agentes, foi construído um ficheiro contendo um conjunto de conversas representativas entre o utilizador e o sistema. O ficheiro com as conversas utilizadas na avaliação está disponível através do link indicado na bibliografia, sob título "*Arquivo de conversas com o assistente hospitalar*" [20]. Este documento com diálogos foi posteriormente enviado para um modelo de linguagem externo, utilizado como avaliador automático,

com o objetivo de analisar o desempenho da solução de forma sistemática. No *prompt* de avaliação foi fornecido o contexto de que o *chat* corresponde a um sistema multi-agente implementado em n8n, concebido como assistente hospitalar para agendamento de cirurgias com base num modelo de otimização, consulta e alteração de dados das listas de espera e dos agendamentos, suportado por um sistema RAG. O *prompt* completo utilizado na LLM avaliadora encontra-se no [Apêndice A](#).

Para cada conversa, a LLM avaliadora foi instruída a atribuir uma pontuação numa escala de 1 a 100, com base em critérios definidos na Tabela I.

TABELA I

CrITÉRIOS de avaliação do agente hospitalar (escala 1 - 100 para cada métrica)

Métrica	Descrição
Precisão	Quão corretas e completas são as respostas face ao pedido e aos dados disponíveis
Coerência conversacional	Encadeamento lógico e manutenção do contexto
Uso do RAG	Capacidade de consultar e atualizar informação relevante corretamente
Robustez	Capacidade de lidar com imprecisões e complexidade dos pedidos
Clareza	Clareza textual e organização da resposta
Segurança operacional	Confirmações, prevenção de erros e consistência operacional

Para além deste julgamento qualitativo, foram ainda recolhidas medidas de tempo de resposta por conversa, permitindo relacionar a qualidade percecionada com o desempenho temporal do sistema.

Resultados obtidos

O documento com as conversas foi disponibilizado aos modelos ChatGPT [21], Gemini [22] e Claude [23], juntamente com o *prompt*, para avaliação das interações. Os resultados globais obtidos estão apresentados na Tabela II.

TABELA II

Desempenhos globais dos modelos ChatGPT, Gemini e Claude nas métricas de avaliação do assistente de agendamento hospitalar (valores entre 1 e 100). A linha “Média” representa a média dos três modelos em cada métrica.

Modelo	Precisão	Coerência	Uso do RAG	Robustez	Clareza	Segurança	Qualidade Geral
ChatGPT	78.9	78	79.6	73.3	74.5	81.9	77.4
Gemini	82.5	82.25	86.25	76.9	86.25	86.25	82.75
Claude	64	65	63	62	65	70	64
Média	75.13	75.1	76.28	70.73	75.25	79.38	74.72

De forma geral, observa-se uma avaliação positiva do sistema, embora com diferenças relevantes entre os modelos avaliadores. O Gemini tende a obter os melhores resultados médios, em particular no uso do RAG, na clareza e na segurança, enquanto o ChatGPT se aproxima destes valores. O Claude apresenta uma avaliação mais conservadora, com desempenhos consistentemente mais baixos.

Análise dos erros ou falhas

A arquitetura encontra-se num estado de protótipo avançado, com a infraestrutura multi-agente operacional e os fluxos principais a funcionar, mas ainda apresenta limitações relevantes.

Observa-se dificuldade em lidar com pedidos imprecisos ou demasiado complexos, uma lógica de mutação de dados frágil, bem como episódios de alucinação. Adicionalmente, verificam-se problemas recorrentes na estrutura do output: mesmo perante instruções explícitas sobre o formato esperado, as LLMs por vezes devolvem respostas desformatadas, exigindo validação e pós-processamento adicionais. A partir dos registos de teste, observam-se tempos de resposta elevados, situados aproximadamente entre 3 e 8 minutos por pergunta em alguns cenários, o que constitui uma limitação crítica para o uso em contexto clínico real.

Em síntese, apesar de o agente evidenciar potencial, os erros de abstração e a latência elevada precisam de ser resolvidos antes de qualquer utilização em ambiente hospitalar.

Trabalhos Futuros

Como trabalho futuro, considera-se relevante investir na melhoria da infraestrutura computacional, nomeadamente através da utilização de servidores mais potentes, que permitam reduzir os tempos de processamento e suportar a adoção de modelos de linguagem mais avançados. Esta evolução poderá contribuir para respostas mais robustas, maior capacidade de raciocínio e melhor desempenho global do sistema em cenários de maior complexidade.

Adicionalmente, poderá ser aprofundada a estruturação da arquitetura multiagente, através da definição de novos agentes especializados em tarefas como validação, verificação e reformulação de respostas, reforçando a fiabilidade do sistema e a qualidade da informação apresentada ao utilizador. Este tipo de especialização funcional tem potencial para melhorar os fluxos de decisão e tornar o comportamento do sistema mais consistente perante pedidos ambíguos ou mais complexos.

Por fim, será importante envolver utilizadores reais, nomeadamente profissionais de saúde e administrativos, em futuras avaliações de usabilidade e aceitação. A inclusão destes perfis permite validar o sistema em contexto mais próximo da prática hospitalar, recolhendo opiniões sobre utilidade, clareza e precisão das respostas, facilidade de utilização e adequação do sistema às necessidades reais dos seus potenciais utilizadores.

Conclusão

Ao longo deste projeto foi desenvolvida uma solução baseada em Agentic AI para apoio ao agendamento hospitalar de forma inteligente, apoiada numa arquitetura multiagente capaz de interpretar pedidos em linguagem natural e convertê-los em solicitações estruturadas do contexto da gestão hospitalar. A solução proposta permitiu integrar diferentes funcionalidades num único sistema conversacional, incluindo a criação de agendamentos com recurso a um modelo de otimização, a consulta e alteração de informação relativa a listas de espera e agendamentos, bem como a análise de dados relevantes para apoio à decisão.

Os resultados obtidos demonstram o potencial deste tipo de abordagem para simplificar a interação com sistemas hospitalares complexos, reduzindo a necessidade de operações manuais e facilitando o acesso à informação por parte dos utilizadores. A utilização de agentes especializados revelou-se adequada para organizar responsabilidades distintas dentro do sistema, tornando-o mais extensível e mais alinhado com a complexidade dos processos reais de agendamento hospitalar. Apesar das limitações identificadas, nomeadamente ao nível da infraestrutura disponível e da necessidade de avaliações futuras com utilizadores reais, o trabalho desenvolvido constitui uma base sólida para a evolução de assistentes inteligentes aplicados ao contexto da saúde.

Em termos globais, o projeto demonstra que a combinação entre modelos de linguagem, sistemas multiagente, RAG e modelos de otimização pode representar uma abordagem promissora para apoiar tarefas administrativas e operacionais no setor hospitalar. Mais do que automatizar respostas, a solução procura criar um mecanismo de intermediação inteligente entre os profissionais e os sistemas de apoio ao agendamento, promovendo maior eficiência, melhor compreensão da informação e apoio mais estruturado à tomada de decisão.

Referências

- [1] Amazon Web Services, “What is agentic AI?,” *AWS*, s.d. [Online]. Disponível em: <https://aws.amazon.com/what-is/agentic-ai/>. Acedido em: 16 abr. 2026.
- [2] Health News, “Listas de espera no SNS agravam-se em 2025, com mais de um milhão à espera de consulta,” 11 fev. 2026. [Online]. Disponível em: <https://healthnews.pt/2026/02/11/401866/>. Acedido em: 18 mar. 2026.
- [3] Saudebemestar.pt, “Inteligência artificial na saúde,” s.d. [Online]. Disponível em: <https://www.saudebemestar.pt/pt/blog-saude/inteligencia-artificial-em-saude/>. Acedido em: 10 mar. 2026.
- [4] C. Stryker, “What are large language models (LLMs)?,” *IBM Think*, s.d. [Online]. Disponível em: <https://www.ibm.com/think/topics/large-language-models>. Acedido em: 16 abr. 2026.
- [5] V. Silva, I. Coelho, e M. Castro, “Ciência de Dados na Gestão Hospitalar: Uma Revisão Sistemática de Aplicações com Inteligência Artificial,” *Anais do SBPO 2025*, out. 2025. doi: 10.59254/sbpo-2025-212153. [Online]. Disponível em: <https://proceedings.science/sbpo/sbpo-2025/trabalhos/ciencia-de-dados-na-gestao-hospitalar-uma-revisao-sistemica-de-aplicacoes-com?lang=pt-br>. Acedido em: 18 mar. 2026.
- [6] PostgreSQL, “PostgreSQL: The World's Most Advanced Open Source Relational Database,” s.d. [Online]. Disponível em: <https://www.postgresql.org/>. Acedido em: 22 abr. 2026.
- [7] n8n, “Welcome to n8n Docs,” s.d. [Online]. Disponível em: <https://docs.n8n.io/>. Acedido em: 18 mar. 2026.
- [8] Ollama, “Ollama's documentation,” s.d. [Online]. Disponível em: <https://docs.ollama.com/>. Acedido em: 22 abr. 2026.
- [9] Streamlit, “Streamlit documentation,” s.d. [Online]. Disponível em: <https://docs.streamlit.io/>. Acedido em: 22 abr. 2026.
- [10] I. Smalley, e S. Susnjara, “O que é uma máquina virtual (VM)?,” *IBM Think*, s.d. [Online]. Disponível em: <https://www.ibm.com/br-pt/think/topics/virtual-machines>. Acedido em: 22 abr. 2026.
- [11] teclaDigital, “UI (User Interface – Interface do Usuário),” s.d. [Online]. Disponível em: <https://tecladigital.pt/glossario/ui-user-interface-interface-do-usuario/>. Acedido em: 23 abr. 2026.
- [12] R. Honório, “O que é o n8n e como ele pode automatizar suas tarefas,” *Asimov Academy*, 14 nov. 2024. [Online]. Disponível em: <https://hub.asimov.academy/blog/o-que-e-n8n-automatizacao/>. Acedido em: 23 abr. 2026.
- [13] GitHub, “GitHub,” s.d. [Online]. Disponível em: <https://github.com/>. Acedido em: 23 abr. 2026.

- [14] Hostinger, “O que é Ollama? Como funciona, principais características e modelos,” 10 mar. 2026. [Online]. Disponível em: <https://www.hostinger.com/pt/tutoriais/o-que-e-ollama>. Acedido em: 23 abr. 2026.
- [15] Amazon Web Services, “O que é engenharia de prompts?,” *AWS*, s.d. [Online]. Disponível em: <https://aws.amazon.com/pt/what-is/prompt-engineering/>. Acedido em: 11 maio 2026.
- [16] OVHcloud, “O que é JSON?,” s.d. [Online]. Disponível em: <https://www.ovhcloud.com/pt/learn/what-is-json/>. Acedido em: 11 maio 2026.
- [17] N. Selvaraj, “O que é Retrieval Augmented Generation (RAG)?,” *DataCamp*, 14 mar. 2025. [Online]. Disponível em: <https://www.datacamp.com/pt/blog/what-is-retrieval-augmented-generation-rag>. Acedido em: 11 maio 2026.
- [18] Supabase, “Build in a weekend Scale to millions,” s.d. [Online]. Disponível em: <https://supabase.com/>. Acedido em: 11 maio 2026.
- [19] J. Liu *et al.*, “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,” in *Proc. 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. [Online]. Disponível em <https://aclanthology.org/2023.emnlp-main.153/>. Acedido em: 28 maio 2026.
- [20] “Arquivo de conversas com o assistente hospitalar,” Google Drive, 28 maio 2026. [Online]. Disponível em: https://drive.google.com/file/d/1m6N4kSR6QejyIFi5luPaDfBzZJW7m_LS/view?usp=sharing. Acedido em: 28 maio 2026.
- [21] “Conversa com ChatGPT para avaliação do assistente hospitalar,” ChatGPT, 28 maio 2026. [Online]. Disponível em: <https://chatgpt.com/share/6a182056-b374-83eb-a00a-1dec9368d5df>. Acedido em: 28 maio 2026.
- [22] “Conversa com Gemini para avaliação do assistente hospitalar,” Gemini, 28 maio 2026. [Online]. Disponível em: <https://gemini.google.com/share/e0e3844f8ecd>. Acedido em: 28 maio 2026.
- [23] “Conversa com Claude para avaliação do assistente hospitalar,” Claude, 28 maio 2026. [Online]. Disponível em: <https://claude.ai/share/77b95cae-70c6-4782-9568-f05b42127f95>. Acedido em: 28 maio 2026.
- [24] K. Guimarães, “pcd-site,” s.d. [Online]. Disponível em: <https://github.com/Kassia-Guimaraes/pcd-site.git>

APÊNDICES

Apêndice A – *Prompt* para avaliação das conversas

O fragmento seguinte apresenta o *prompt* utilizado na LLM externa para avaliar as conversas entre utilizadores e o agente desenvolvido no âmbito deste projeto.

Quero que seja uma LLM-as-a-judge e avalies a capacidade de interpretar pedidos em linguagem natural e o desempenho de resposta, assim como os tempos de resposta. Quero que faças a análise para cada conversa. Contexto: o chat é um multi-agente criado no n8n, com o intuito de ser um assistente hospitalar, que realiza agendamentos através de um modelo de otimização, consulta e realiza alterações em dados da lista de espera e dos agendamentos previamente realizados e armazenados num sistema RAG. Classifica cada conversa do agente numa escala de 1 a 100, considerando:

- Precisão
- Coerência Conversacional
- Uso do RAG
- Robustez
- Clareza
- Segurança Operacional
- Qualidade Geral